

Experiment – 4

Name: Simranjeet Kaur

UID: 24BAI70048

Branch: CSE – AIML (CS221)

Section/Group: 24AIT_NTPP-2A

Semester: 5th

Date of Performance: 31/07/2026

Subject: Full Stack-II

Subject Code: 24CSP-337

EXPERIMENT – 4.1

1. **AIM:** To design and implement an interactive calendar interface for scheduling and managing posts.

2. **OBJECTIVE:**

1. To understand time-based data visualization in UI systems
2. To implement calendar-based scheduling interfaces
3. To map structured data to temporal layouts

3. **SOFTWARE REQUIREMENTS:**

1. Windows/Linux/macOS
2. Node.js (LTS Version)
3. NPM (Node Package Manager)
4. Visual Studio Code
5. Google Chrome / Microsoft Edge
6. React.js
7. Command Prompt / Terminal
8. Redux

4. **THEORY**

An interactive calendar is a graphical user interface (GUI) component used to display and manage events based on dates and time. It helps users organize schedules by presenting information in a clear and visual format. In a post scheduling system, each post is treated as an event and is assigned to a specific date or time slot on the calendar. In React applications, libraries such as FullCalendar are commonly used to create interactive calendars. These libraries provide features like month, week, and day views, event display, date selection, and drag-and-drop functionality for rescheduling events. React's state management allows the calendar to update dynamically whenever a new post is added, edited, or deleted. Interactive calendars improve the user experience by making content planning simple, reducing scheduling conflicts, and enabling efficient management of time-based data. They are widely used in social media schedulers, event management systems, booking applications, and task management software.

5. **IMPLEMENTATION/PROCEDURE**

1. Create a new React project and install the required FullCalendar packages.
2. Design the calendar interface using the FullCalendar component
3. Create an array of post events containing the post title and scheduled date.
4. Render the events dynamically on the calendar..
5. Implement date click functionality to allow users to add new posts.
6. Enable event interactions such as drag-and-drop for rescheduling posts.
7. Connect the calendar with React state to update events dynamically.
8. Apply CSS styling to create an attractive and responsive user interface.
9. Run the application and verify that posts are displayed and updated correctly on the calendar.
10. Test all functionalities such as adding, viewing, and rescheduling posts to ensure proper working.

6. CODE

```
import './App.css';
import CalendarComponent from './CalendarComponent';
function App() {
  return (
    <div className="app">
      <div className="header">
        <h1><img alt="Calendar icon" data-bbox="191 231 211 241"/> Post Scheduler</h1>
        <p>Plan, schedule and manage your social media posts</p>
      </div>
      <div className="calendar-card">
        <CalendarComponent />
      </div>
    </div>
  );
}
```

export default App;

7. OUTPUT

The output displays an interactive Post Scheduler developed using React.js and FullCalendar. The application provides a monthly calendar view where scheduled posts are displayed on their respective dates. Users can click on any date to add a new post, and existing posts can be rescheduled using drag-and-drop functionality. The interface features a responsive design with a styled header and calendar layout, making it easy to organize, view, and manage social media posts efficiently.

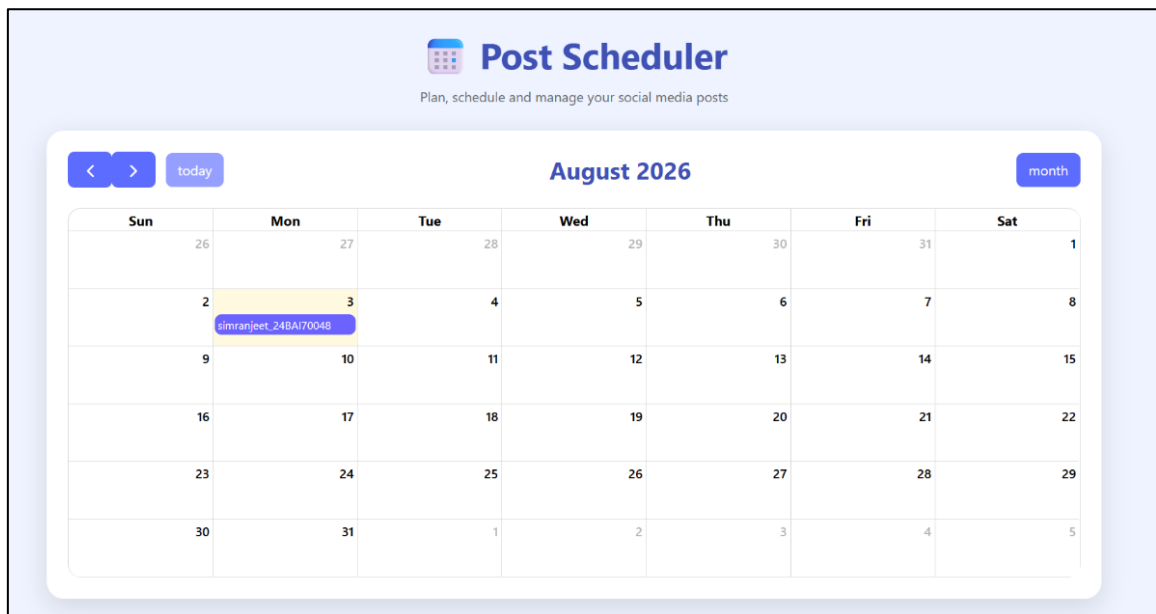


Fig 4.1: Interactive Post Scheduler Calendar Interface

8. LEARNING OUTCOME

- Design and implement an interactive calendar interface using React.js.
- Integrate the FullCalendar library into a React application.
- Map post data to calendar events and display them dynamically.
- Implement user interactions such as adding and rescheduling events.
- Manage calendar data using React state
- Develop a responsive and user-friendly scheduling interface.

EXPERIMENT –4.2

1. **AIM:** To optimize rendering performance and implement testing strategies for interactive UI components.

2. **OBJECTIVE:**

1. To understand performance bottlenecks in UI systems
2. To implement testing for UI components and logic
3. To optimize rendering using memoization techniques

3. **SOFTWARE REQUIREMENTS:**

1. Windows/Linux/macOS
2. Node.js (LTS Version)
3. npm (Node Package Manager)
4. Visual Studio Code
5. Google Chrome / Microsoft Edge
6. React.js
7. Command Prompt / Terminal

4. **THEORY**

Interactive web applications such as dashboards, calendars, and task management systems frequently update their user interface based on user actions. These frequent updates can cause unnecessary component re-rendering, leading to slower performance and increased memory usage. React provides built-in optimization techniques to improve rendering efficiency and enhance the user experience. In this experiment, performance optimization is achieved using `React.memo`, `useMemo`, and `useCallback`. `React.memo` prevents a component from re-rendering when its props have not changed, `useMemo` caches the results of expensive computations such as filtering tasks, and `useCallback` memoizes event handler functions to maintain stable references across renders. Together, these techniques reduce unnecessary rendering and improve application responsiveness.

The application is also tested using Jest and React Testing Library to verify that UI components function correctly and respond properly to user interactions. This experiment demonstrates how performance optimization and testing contribute to building fast, reliable, and maintainable React applications.

5. **IMPLEMENTATION/PROCEDURE**

1. Create a new React project and install the required dependencies.
2. Develop a Smart Task Dashboard with features to add, search, complete, undo, and delete tasks.
3. Create a reusable TaskCard component to display individual tasks.
4. Apply `React.memo` to the TaskCard component to prevent unnecessary re-renders.
5. Use `useMemo` to optimize task filtering during search operations.
6. Use `useCallback` to memoize event handler functions such as add, delete, and complete task.
7. Style the application using CSS to create an interactive and user-friendly interface.
8. Write unit tests using Jest and React Testing Library to verify component rendering and user interactions.
9. Run the application in the browser and test all functionalities.

6. **CODE**

```
import Dashboard from "./Dashboard";
import "./App.css";
```

```
function App() {
  return (
    <div>
      <Dashboard />
    </div>
  );
}
```

export default App;

7. OUTPUT

The figure shows the Smart Task Dashboard developed using React.js. The application allows users to add, search, complete, undo, and delete tasks through an interactive interface. It displays the total number of tasks and updates the task status dynamically based on user interactions. The dashboard is optimized using React.memo to prevent unnecessary component re-rendering, useMemo to efficiently filter tasks during search operations, and useCallback to maintain stable function references for event handlers. These optimization techniques improve rendering performance and provide a responsive user experience. The application is also suitable for unit testing using Jest and React Testing Library, ensuring the correctness and reliability of the interactive UI components.

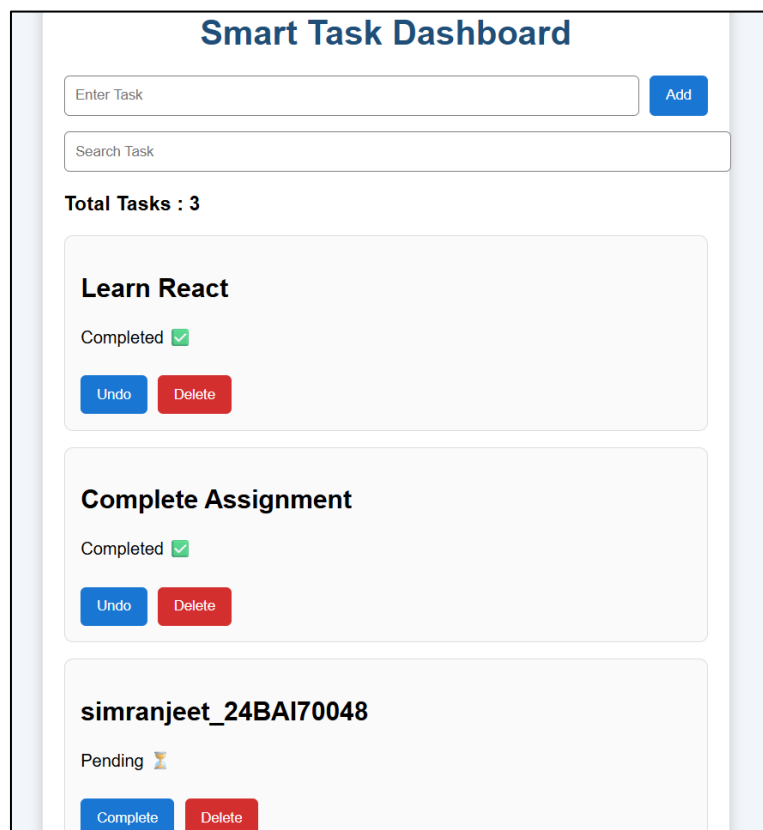


Fig 4.2: Smart Task Dashboard – Interactive React UI with Performance Optimization.

8. LEARNING OUTCOME

- Understand the importance of performance optimization in React applications.
- Apply React.memo, useMemo, and useCallback to reduce unnecessary component re-renders.
- Develop interactive UI components with efficient state management.
- Write and execute unit tests using Jest and React Testing Library.
- Implement dynamic UI rendering according to the logged-in user's role.
- Analyze and improve the performance, responsiveness, and reliability of React-based user interfaces.
- Identify performance bottlenecks and unnecessary component re-renders